



Berachain Beacon Kit #1

Defensive Fuzzing Report

Feb. 3, 2025

Prepared By:

0xScourgedev | Lead Fuzzing Specialist

0xscourgedev@perimetersec.io

Rappie | Lead Fuzzing Specialist

rappie@perimetersec.io

Table of Contents

About Perimeter	3
Risk Classification	4
Services Provided	5
Files in Scope	6
Methodology	7
Issues	8
Invariants	9
Disclaimer	11

About Perimeter

Perimeter's mission is to deliver the highest quality fuzzing services to protocols by uniting the world's foremost fuzzing specialists. We possess extensive expertise in fuzzing a diverse range of protocols, from smaller, niche protocols to some of the largest and most complex in DeFi.

In order to deliver on our mission, we have developed the most advanced scaffolding and libraries, enabling us to create highly sophisticated fuzzing suites tailored to meet the unique challenges of each protocol.

Learn more about us at perimetersec.io.

Risk Classification

The severity of security issues identified during the security review is classified according to the table below.

- A. Critical findings are highly likely to be exploited with severe impact on the protocol and require immediate attention.
- B. High findings are very likely to occur, easy to exploit, or difficult but highly incentivized, and should be resolved as quickly as possible.
- C. Medium findings are possible in certain circumstances or when incentivized, with a moderate likelihood of occurring, and should be addressed.
- D. Low findings involve rare circumstances to exploit or offer little to no incentives, though addressing them is still recommended.
- E. Informational issues represent improvements that do not impact the project's overall security but are worth considering.

Severity Level	High Impact	Medium Impact	Low Impact
High Likelihood	Critical	High	Medium
Medium Likelihood	High	Medium	Low
Low Likelihood	Medium	Low	Low

Services Provided

Perimeter has successfully delivered a comprehensive suite of services that include:

- **Fuzzing Suite Development:** Design and implement a stateful fuzzing suite using Medusa. This suite will be tailor-made for the protocol and contracts in scope. The completed fuzzing suite can later be integrated into the testing suite to serve long-term security needs.
- **Findings Reporting:** We provided thorough documentation and reporting of all findings identified throughout the engagement.
- **Invariant Testing Assurance:** Guarantee that each invariant implemented will be tested no fewer than 10,000,000 instances, ensuring thorough validation and reliability.
- **Proof-of-Concept Development:** Develop a corresponding Proof-of-Concept (PoC) for each finding and assertion/property counterexample identified, to demonstrate potential vulnerabilities and their implications.
- **Comprehensive Final Report:** Create a detailed final report that will include all findings, along with their corresponding PoCs. This report will also detail the invariants tested, their run status, and the number of runs, providing a comprehensive overview of the engagement's outcomes.

Files in Scope

The engagement will be focused on the files listed below, acquired from commit [dd024c5b196afe43cf871b5f825a7e371fc4542e](https://github.com/0x00/commit/dd024c5b196afe43cf871b5f825a7e371fc4542e).

```
src
├── eip4788
│   ├── BeaconVerifier.sol
│   ├── interfaces
│   │   └── IBeaconVerifier.sol
│   ├── SSZ.sol
│   └── Verifier.sol
```

The proofs were generated using the Beacon Kit Go code, which brought many files partially or fully into scope. Given the size of the codebase, specific filenames are not listed.

Files Out of Scope

Files outside the scope were not directly considered in achieving the target. However, since many of these files are utilized by those within the scope, a significant portion was indirectly covered.

Methodology

The primary goal of this engagement was to rigorously test proof validation and to establish a robust foundation that can be easily extended in the future. To maintain consistency with the Beacon Kit code, we utilized many of its components, written in Golang, for the proof generation script. These proofs were then verified using the proof validation Solidity contracts.

The main threats under investigation were false positive cases, false negative cases, and potential denial-of-service vulnerabilities.

The following parameters were randomized during proof generation:

- Validator public keys
- The number of validators
- Selected balances
- Specific slashing events
- Various roots

Due to the use of arbitrary execution, the effectiveness of coverage-guided fuzzing was significantly limited. As a result, we employed only Medusa to maximize the number of test runs.

Issues

At Perimeter, our objective is to thoroughly investigate and uncover critical vulnerabilities through fuzzing and reveal issues often overlooked in manual reviews. Any lower-severity findings are incidental to this core focus.

Severity	Count	Fixed	Partially Resolved	Acknowledged
Critical	0	0	0	0
High	0	0	0	0
Medium	0	0	0	0
Low	0	0	0	0
Informational	0	0	0	0
Total	0	0	0	0

Invariants

We created many tests to verify the correctness of **12** invariants described in the table below. During the execution phase, these invariants were assessed for a total of **32,000,000+** calls.

Note: Due to the usage of arbitrary execution, the number of runs is significantly less than a typical invariant suite.

The table below lists all invariants that are part of this engagement.

Invariant	Description	Tested	Passed	# Runs
PROOF-01	If the zeroValidatorPubkeyGIndex is different, the proof should never be valid	✓	✓	32M+
PROOF-02	If the executionNumberGIndex is different, the proof should never be valid	✓	✓	32M+
PROOF-03	If the executionFeeRecipientGIndex is different, the proof should never be valid	✓	✓	32M+
PROOF-04	If the proof for verifyBeaconBlockProposer was not modified post-generation, then the proof should always be valid	✓	✓	32M+
PROOF-05	If the proof for verifyExecutionNumber was not modified post-generation, then the proof should always be valid	✓	✓	32M+
PROOF-06	If the proof for verifyExecutionFeeRecipient was not modified post-generation, then the proof should always be valid	✓	✓	32M+
PROOF-07	If the zeroValidatorPubkeyGIndex is the same and the proof for verifyBeaconBlockProposer was modified post-generation, then the proof should never be valid	✓	✓	32M+
PROOF-08	If the executionNumberGIndex is the same and the proof for verifyExecutionNumber was modified post-generation, then the proof should never be valid	✓	✓	32M+

Invariant	Description	Tested	Passed	# Runs
PROOF-09	If the executionFeeRecipientGIndex is the same and the proof for verifyExecutionFeeRecipient was modified post-generation, then the proof should never be valid	✓	✓	32M+
REV-01	setZeroValidatorPubkeyGIndex never reverts	✓	✓	32M+
REV-02	setExecutionNumberGIndex never reverts	✓	✓	32M+
REV-03	setExecutionFeeRecipientGIndex never reverts	✓	✓	32M+

Disclaimer

All activities conducted by Perimeter in connection with this project were carried out in accordance with the terms outlined in a Statement of Work and an agreed-upon project plan, as set forth in a proposal document delivered prior to the commencement of the project.

Security assessment projects are subject to time limitations, and as such, the findings presented in this report should not be interpreted as an exhaustive or comprehensive identification of all security issues, vulnerabilities, or defects within the target codebase. Perimeter makes no representations or warranties that the target codebase is free from defects.

Furthermore, this report is not intended to be, and should not be construed as, investment advice or a recommendation to participate in any financial transactions. The content herein does not constitute endorsements or recommendations for any financial decisions, securities, or investment strategies.